Guía paso a paso: Tienda con Laravel y Docker

26/10/2026

Antes de empezar

Esta guía te lleva de una tabla vacía a una página `/productos` con precios en guaraníes, y deja el proyecto listo para entregar en GitHub.

Laravel reparte el trabajo en piezas. Cada paso de esta guía toca una de ellas:

Pieza	Para qué sirve	Archivo
Migración	Define las columnas de la tabla	`database/migrations/..._create_productos_table.php`
Modelo	Representa la tabla en PHP (`Producto` es la tabla `productos`)	`app/Models/Producto.php`
Seeder	Carga datos de ejemplo	`database/seeders/ProductoSeeder.php`
Ruta	Dice qué hacer cuando alguien entra a una URL	`routes/web.php`
Controlador	Busca los datos y se los pasa a la vista	`app/Http/Controllers/ProductoController.php`
Vista (Blade)	El HTML que ve el usuario	`resources/views/productos/index.blade.php`

Requisitos previos:

- Docker Desktop encendido (abajo a la izquierda dice *Engine running*).
- El proyecto `tienda` creado según la clase del 20/10.
- La terminal de Ubuntu abierta (el prompt termina en `$`).

Todos los comandos se escriben desde la carpeta del proyecto:

```
cd ~/proyectos/tienda
docker compose up -d
```

`docker compose up -d` enciende los contenedores `app` (Laravel) y `db` (MySQL). Si ya estaban encendidos, no pasa nada.

Pasos 1 y 2: la tabla y el modelo

La tabla `productos` necesita las columnas `nombre` y `precio`, y el modelo tiene que permitir llenarlas.

Paso 1: la migración

Abrí `database/migrations/2026_10_21_003334_create_productos_table.php`. Hoy solo crea `id` y las fechas. Agregá las dos columnas entre ellas:

```
Schema::create('productos', function (Blueprint $table) {
    $table->id();
    $table->string('nombre');          // texto, ej: "Chipa"
    $table->unsignedInteger('precio'); // entero sin negativos, ej: 10000
    $table->timestamps();
});
```

El guaraní no usa decimales, por eso `precio` es un entero.

Paso 2: el modelo

Abrí `app/Models/Producto.php` y dejalo así:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Producto extends Model
{
    protected $fillable = ['nombre', 'precio'];
}
```

`$fillable` lista las columnas que Laravel permite llenar con `Producto::create([...])`. Sin ella, Laravel bloquea la carga por seguridad.

☐ La migración tiene `nombre` y `precio`
☐ El modelo tiene `$fillable`

Pasos 3 y 4: el seeder y los datos

La tarea pide 5 productos cargados con `migrate --seed`. El seeder los define y `DatabaseSeeder` lo ejecuta.

Paso 3: el seeder

Abrí `database/seeders/ProductoSeeder.php` y dejalo así:

```
<?php

namespace Database\Seeders;

use App\Models\Producto;
use Illuminate\Database\Seeder;

class ProductoSeeder extends Seeder
{
    public function run(): void
    {
        $productos = [
            ['nombre' => 'Cocido',         'precio' => 5000],
            ['nombre' => 'Chipa',          'precio' => 10000],
            ['nombre' => 'Mbejú',          'precio' => 3000],
            ['nombre' => 'Sopa paraguaya', 'precio' => 8000],
            ['nombre' => 'Tereré',         'precio' => 6000],
        ];

        foreach ($productos as $producto) {
            Producto::create($producto);
        }
    }
}
```

Cada línea entre corchetes es un producto. Ojo con dos errores del primer intento: el modelo se llama `Producto` (singular) y cada producto va en su propio array, porque repetir `'nombre'` dentro de un mismo array hace que solo cuente el último.

Ahora abrí `database/seeders/DatabaseSeeder.php`. Es el seeder principal, el que llama a los demás. Dentro de `run()` agregá:

```
$this->call(ProductoSeeder::class);
```

Sin esa línea, `ProductoSeeder` nunca se ejecuta.

Paso 4: crear la tabla y cargar los datos

```
docker compose exec app php artisan migrate:fresh --seed
```

- `docker compose exec app` ejecuta el comando dentro del contenedor de Laravel.
- `migrate:fresh` borra todas las tablas y las crea de nuevo, así toma las columnas nuevas.
- `--seed` carga los datos al terminar.

Para verificar, abrí SQLyog con `127.0.0.1`, puerto `3308`, usuario `root`, clave `secreto` y base `laravel`. Tiene que existir la tabla `productos` con 5 filas.

Si aparece `Connection refused`, MySQL todavía está arrancando: esperá 30 segundos y repetí el comando.

☐ El seeder tiene 5 productos
☐ `DatabaseSeeder` llama a `ProductoSeeder`
☐ La tabla `productos` muestra 5 filas en SQLyog

Pasos 5 a 7: mostrar los productos en /productos

El navegador pide `/productos`, la ruta llama al controlador, el controlador busca los productos en la base y la vista los dibuja.

Paso 5: el controlador

Si usaste `make:model Producto -mcrs`, ya existe `app/Http/Controllers/ProductoController.php`. Si no existe, creálo:

```
docker compose exec app php artisan make:controller ProductoController
```

Abrilo y dejá el método `index()` así. El `use` va arriba, junto a los otros:

```
use App\Models\Producto;

public function index()
{
    $productos = Producto::all(); // trae todas las filas de la tabla
    return view('productos.index', ['productos' => $productos]);
}
```

Paso 6: la ruta

Abrí `routes/web.php` y agregá al final:

```
use App\Http\Controllers\ProductoController;

Route::get('/productos', [ProductoController::class, 'index']);
```

Se lee así: cuando alguien pida `/productos`, ejecutá el método `index` de `ProductoController`.

Paso 7: la vista con Blade

Creá la carpeta `resources/views/productos/` y dentro el archivo `index.blade.php`:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Productos</title>
</head>
<body>
    <h1>Lista de productos</h1>
    <ul>
        @foreach ($productos as $producto)
            <li>
                {{ $producto->nombre }} - Gs. {{ number_format($producto->precio, 0, ',',
'.') }}
            </li>
        @endforeach
    </ul>
</body>
</html>
```

- `{{ ... }}` imprime un valor en la página.
- `@foreach ... @endforeach` repite el bloque una vez por producto.
- `number_format($precio, 0, ',', '.')` usa 0 decimales y puntos de miles, y convierte `5000` en `5.000`.

Para probar, abrí `http://localhost:8080/productos`. Tienen que verse los 5 productos con `Gs.`. Esa es la captura que se entrega.

☐ El controlador tiene `index()` con `Producto::all()`
☐ La ruta `/productos` está en `routes/web.php`
☐ La vista existe en `resources/views/productos/index.blade.php`
☐ `localhost:8080/productos` muestra los 5 productos

Paso 8: agregar phpMyAdmin en el puerto 8081

phpMyAdmin se agrega como un tercer servicio en `compose.yaml`, al mismo nivel que `app` y `db`. En YAML la sangría (los espacios al inicio de cada línea) es parte de la sintaxis, así que `phpmyadmin:` lleva los mismos 2 espacios que `app:` y `db:`.

```
services:
  app:
    ...
  db:
    ...
  phpmyadmin:
    image: phpmyadmin:latest
    ports: ["8081:80"]
    environment:
      PMA_HOST: db
    depends_on: [db]
```

- `PMA_HOST: db` le dice a phpMyAdmin a qué servidor conectarse: el servicio `db`.
- `8081:80` es el puerto 8081 de tu PC conectado al puerto 80 del contenedor.
- Los `...` representan lo que ya tenías en `app` y `db`; no los borres ni los copies literal.

Aplicá el cambio:

```
docker compose up -d
```

Para probar, abrí `http://localhost:8081` con usuario `root` y clave `secreto`. Tiene que aparecer la base `laravel` con la tabla `productos`.

☐ `phpmyadmin` está al mismo nivel que `app` y `db`
☐ `localhost:8081` abre phpMyAdmin

Paso 9: subir a GitHub

El repositorio tiene que llevar `.env.example` y un README, y no puede llevar `.env`, porque ahí están las contraseñas.

9.1 Revisar que `.env` no se suba

Abrí `.gitignore` y comprobá que tenga una línea `.env`. Laravel ya la trae.

9.2 Revisar `.env.example`

Es la copia sin secretos que sí se sube. Las líneas `DB_` tienen que ser las de MySQL, no las de sqlite. Si todavía muestran sqlite, copiá ahí las líneas de `env-docker.txt`.

9.3 Escribir el README.md

Abrí `README.md` y reemplazalo por los comandos para levantar el proyecto:

```
# Tienda

## Levantar el proyecto
cp .env.example .env
docker compose up -d --build
docker compose exec app composer install
docker compose exec app php artisan key:generate
docker compose exec app php artisan migrate --seed

Abrir http://localhost:8080/productos
phpMyAdmin: http://localhost:8081
```

9.4 Crear el repositorio en GitHub

En github.com elegí *New repository*, ponele de nombre `tienda` y dejalo vacío, sin README.

9.5 Subir el código

```
git init
git add .
git commit -m "Tienda con Laravel y Docker"
git branch -M main
git remote add origin https://github.com/TU_USUARIO/tienda.git
git push -u origin main
```

Cambiá `TU_USUARIO` por tu usuario de GitHub. Si es la primera vez, GitHub te pide autenticarte.

9.6 Comprobar

En la página del repositorio, verificá que **no aparezca** `.env` y que sí estén `.env.example` y `README.md`. Copiá el enlace del repositorio para la entrega.

☐ `.env` está en `.gitignore`
☐ `.env.example` tiene las líneas `DB_` de MySQL
☐ El README tiene los comandos
☐ El repositorio no muestra `.env`

Errores comunes y entrega

Si algo falla, buscá el mensaje en esta tabla antes de cambiar nada:

Mensaje o síntoma	Causa	Solución

`Class "Productos" not found`	El modelo se llama `Producto`, en singular	Usar `Producto` en el seeder y en el `use`
`Unknown column 'nombre'`	La tabla se creó antes de agregar la columna	Correr `migrate:fresh --seed` otra vez
`Add [nombre] to fillable property`	Falta `$fillable` en el modelo	Agregar `protected $fillable = ['nombre', 'precio'];`
`Connection refused` o `[2002]`	MySQL todavía está arrancando	Esperar 30 segundos y repetir el comando
Error 500 o `Permission denied`	Apache no puede escribir en `storage`	`chmod -R 777 storage bootstrap/cache`
404 en `/productos`	La ruta no está registrada	Revisar `routes/web.php`
Página en blanco o error 500 sin detalle	Error de PHP en el código	`docker compose exec app tail -n 30 storage/logs/laravel.log`
`port is already allocated`	Otro contenedor usa el puerto	`docker ps` y luego `docker rm -f NOMBRE`

Entrega: martes 27/10

La tarea se entrega con dos cosas:

☐ El enlace al repositorio de GitHub
☐ La captura de `localhost:8080/productos`

Antes de entregar, revisá la consigna completa:

☐ Modelo `Producto` con `nombre` y `precio`
☐ Seeder con 5 productos y `migrate --seed` ejecutado
☐ Lista en `/productos` con Blade y precios como `Gs. 5.000`
☐ phpMyAdmin en el puerto 8081
☐ GitHub con `.env.example`, sin `.env`, y un README con los comandos